

Ejercicios Clases Kotlin

[Descargar estos ejercicios](#)

[Ejercicio 1](#)

[Ejercicio 2](#)

[Ejercicio 3](#)

Ejercicio 1

Composición: Vehículo y Motor

Implementa una relación de **composición** entre las clases **Vehículo** y **Motor** . El motor es parte integral del vehículo y no puede existir sin él.

Ejercicio 1: Composición Vehículo-Motor

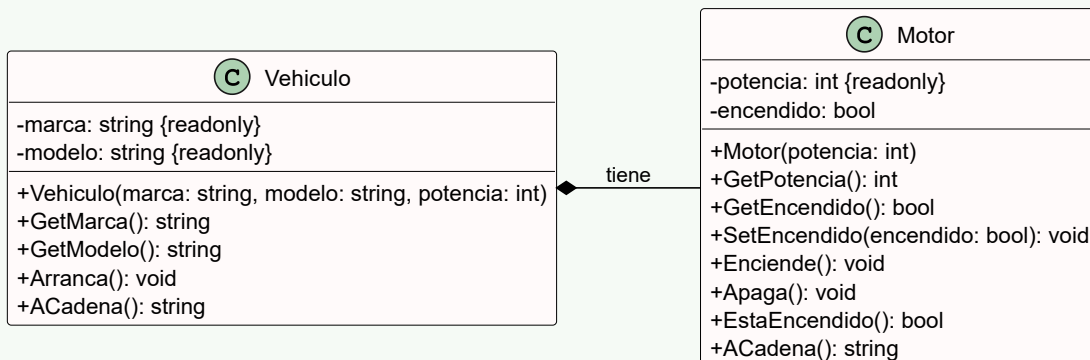
Creando vehículo con motor integrado...

Vehículo: Toyota Corolla
Motor: 120 CV - Estado: Apagado

Arrancando el vehículo...

Vehículo: Toyota Corolla
Motor: 120 CV - Estado: Encendido

Presiona cualquier tecla para salir...



Requisitos:

- Crea las clases necesarias para implementar correctamente el diagrama de clases. Sobre todo, fíjate en la relación entre clases y lo que conlleva.
- En el programa principal, crea las instancias de los objetos necesarias y Muestra el estado inicial como se ve en la salida
- Arrancar el vehículo y muestra el estado del vehículo después de arrancado.

Ejercicio 2

`data class` como Value Objects

Implementa un sistema para gestionar pisos de alquiler usando una `data class` con un **enumerado definido dentro de la propia clase**. El objetivo es demostrar cómo modificar una o dos propiedades con `copy(...)` manteniendo el resto intacto.

Ejercicio 2: Piso de alquiler con `data class` y `enum` interno

```
=== DATA CLASS PisoAlquiler ===
Creando piso base...
Piso original: Piso #101 en Calle Mayor 12 | ESTUDIO | 850.0€/mes | Estado: Disponible

--- Modificando con copy(...) ---
Piso con nuevo precio: Piso #101 en Calle Mayor 12, 3A | ESTUDIO | 920.0€/mes | Estado: Disponible
Piso actualizado (tipo y alquilado): Piso #101 en Calle Mayor 12, 3A | ATICO | 920.0€/mes | Estado: Disponible

--- Tipo de inmueble ---
ESTUDIO como [Microestudio, Ideal para una persona]
PISO como [Apartamento pequeño, Perfecto para parejas]
DUPLEX como [Dos plantas, Más espacio para familias]
ATICO como [Vistas panorámicas, Terraza grande]
```

Requisitos:

- Crea una `data class` llamada `PisoAlquiler` con las propiedades:
 - `codigo: Int`
 - `direccion: String`
 - `precioMensual: Double`
 - `tipo: TipoPiso`
 - `alquilado: Boolean`
- La `data class` debe definir internamente un **enumerado** llamado `TipoPiso`.
 - El enumerado tendrá un constructor con dos propiedades:
 - `codigo: Int`
 - `ejemplos: List<String>`
 - Los valores del enumerado serán:
 - `ESTUDIO(10, listOf("Microestudio", "Ideal para una persona"))`
 - `PISO(20, listOf("Apartamento pequeño", "Perfecto para parejas"))`
 - `DUPLEX(30, listOf("Dos plantas", "Más espacio para familias"))`
 - `ATICO(40, listOf("Vistas panorámicas", "Terraza grande"))`

- Sobrescribe el método `toString()` del enumerado para que muestre algo como:

```
ESTUDIO como [Microestudio, Ideal para una persona]
```

- Además la data class tendrá otra funcionalidad a través del método `resumenAlquiler()` que permitirá devolver una línea con el formato que puedes ver en la salida, esta función llamará al `toString` del enumerado
- En el programa principal:
 - Crea un **piso base** con su tipo correspondiente.
 - Modifica el precio mensual usando `copy(...)`.
 - Modifica también el tipo y el estado de alquilerado usando otra copia.
 - Muestra la salida con los cambios realizados.



Ayuda

La idea es que un objeto `PisoAlquiler` se convierta en otra versión distinta sin cambiar el original, usando `copy(...)` cuando quieras actualizar uno o dos campos como el precio y el estado de disponibilidad.

Ejercicio 3

Patrón Sum Type: Resultado de Pago

Estás desarrollando el **módulo de cobros de una tienda online**. Dependiendo del monto de la compra y del estado del sistema, un pago puede tener tres resultados posibles, y cada uno lleva información diferente.

Debes implementar el **patrón Sum Type** para modelar el resultado de una transacción y un objeto que procese dichos pagos.

Especificaciones:

1. Crea una sealed interface llamada `ResultadoPago`.
2. Definir las Variantes (Los Casos): Crea tres **data classes** anidadas que implementen

`ResultadoPago`:

- `Aprobado`: Debe contener un `UUID` `codigoTransaccion`.
- `Rechazado`: Debe contener un `String` `motivo`.
- `EnRevision`: (Para montos sospechosos) Debe contener un `Int` `nivelRiesgo` (ej. de 1 a 5).

3. Crea un `object` llamado `PasarelaDePago` con la función:

```
fun procesar(monto: BigDecimal): ResultadoPago
```

Implementa la siguiente lógica simulada:

- **Si el monto es menor a cero**: Retorna `Rechazado` con motivo `"Monto inválido"`.
- **Si el monto es mayor a 5000**: Retorna `EnRevision` con nivel de riesgo `5`.
- **Si el monto es mayor a 1000**: Retorna `EnRevision` con nivel de riesgo `1`.
- **En cualquier otro caso**: Retorna `Aprobado` con un identificador generado aleatoriamente (`UUID.randomUUID()`).

4. En la función `main`:

- Crea una lista inmutable (`listOf`) de montos de prueba (`BigDecimal`): -50, 100, 2500 y 6000.
- Itera sobre la lista procesando cada pago con `PasarelaDePago.procesar(monto)`.
- Utiliza una expresión `when` exhaustiva (Pattern Matching / Smart Casts) sobre el resultado para formatear e imprimir un mensaje en consola según el tipo, accediendo directamente a sus propiedades:
 - `ResultadoPago.Aprobado`:
`" [OK] Pago exitoso. ID: ${resultado.codigoTransaccion} "`
 - `ResultadoPago.Rechazado`: `" [X] Error en el pago. Motivo: ${resultado.motivo} "`

- **ResultadoPago.EnRevision :**

" [?] *El pago requiere auditoría. Riesgo: \${resultado.nivelRiesgo}* "

- Muestra por consola el monto formateado en moneda junto con el mensaje resultante.